Event Collision Detector: Predictive Launch-Window Intelligence

Project Proposal for UCS503P

Thapar Institute of Engineering and Technology

Student Names	Jaskaran Singh, Vaani Mehta, Parmeet Singh
Roll Numbers	1024160033, 1024160123, 1024160021
Course	Software Engineering (UCS503P)
Submitted to	Jeelani Asif

1 Higher-order goal

The project aims to improve the success of high-stakes launches by reducing avoidable competition for audience attention, media coverage, and marketing visibility. The immediate engineering objective of the **Event Collision Detector** is to convert cross-industry event intelligence into an actionable scheduling decision by aggregating external events, evaluating a proposed launch window, generating an explainable collision-risk score, and recommending lower-risk alternatives.

2 Time-to-value

- Deliver an initial single-industry event ingestion pipeline and heuristic collision score within the first iteration.
- Prioritize fast feedback using existing event APIs and explainable rule-based scoring before introducing advanced predictive models.
- Define first-iteration success as a complete date input → event retrieval → risk score → recommendation flow.

3 Problem Statement

Marketing and product teams often choose launch dates using internal schedules without sufficient visibility into the external event landscape.

- **Blind-Spot Scheduling:** Internal calendars do not show competitor launches, industry conferences, or major public events.
- **Diluted Audience Attention:** Overlapping events fragment media coverage and audience engagement.
- **Sunk Marketing Spend:** A poorly timed launch can reduce the return on months of campaign preparation.

- **No Systematic Alternative-Date Discovery:** Teams lack a structured way to identify nearby low-risk windows.

4 Proposed Solution

4.1 Overview

Build a web-based **Event Collision Detector** with the following components:

- **Cross-Industry Event Radar:** Aggregates external events from APIs, conference schedules, and selected public sources.
- **Real-Time Collision Scoring:** Produces a High / Medium / Low risk score for a proposed date.
- **Optimal Launch Recommendations:** Searches surrounding dates and returns lower-risk alternatives.
- **Interactive Risk Dashboard:** Displays the risk score, conflicting events, and explanations.
- **Event and Source Management:** Allows administrators to maintain event records and tracked sources.

4.2 Core workflow

1. Event organizer enters a proposed launch date, industry, target audience, and event details.
2. Backend retrieves nearby competing events from the event store.
3. Collision engine compares timing, relevance, audience overlap, and estimated media friction.
4. System assigns a risk level and identifies the events contributing to it.
5. Recommendation engine scans nearby dates and returns lower-risk alternatives.
6. Dashboard displays and stores the final collision report.

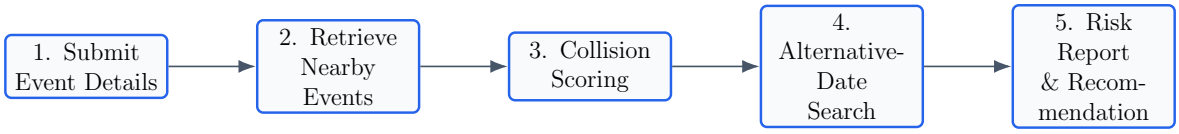

Figure 1: Event Collision Detector Core Workflow Execution Pipeline

4.3 Operational constraints for an educational setting

- Limit initial event coverage to selected industries such as Technology and Entertainment.
- Prefer existing APIs and structured public feeds over large-scale scraping infrastructure.
- Use heuristic, explainable scoring so that each risk score can be traced to specific events.
- Keep the system deployable on standard cloud infrastructure or containerized local environments.

5 Solution Approach

This project focuses on software engineering, event ingestion, low-latency retrieval, explainable scoring, and a responsive dashboard rather than novel machine-learning research.

5.1 Collision analysis logic

- **Temporal Proximity:** Events closer to the proposed date receive greater weight.
- **Industry Relevance:** Same-industry or related events contribute more strongly to risk.
- **Audience Overlap:** Similar target-audience tags increase attention conflict.
- **Event Magnitude:** Larger events receive greater importance.
- **Media Friction:** Events competing for similar channels or press attention increase the score.

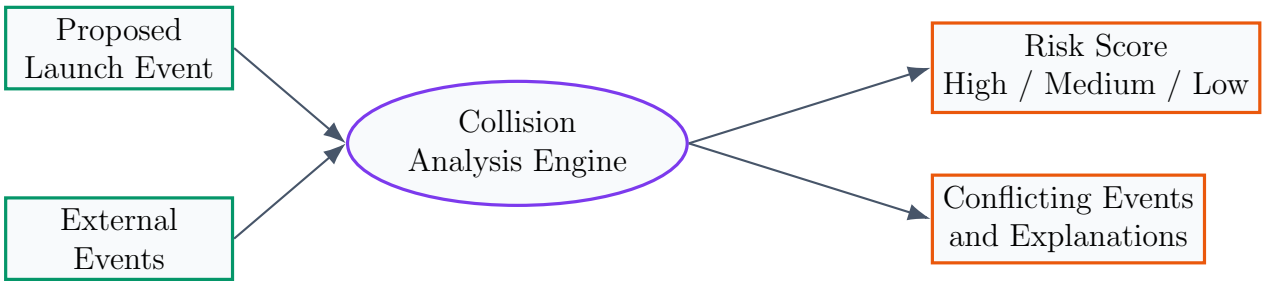

Figure 2: Collision Analysis and Explainable Risk-Scoring Logic

5.2 System architecture

A layered architecture separates user interaction, application logic, scoring, ingestion, and storage.

- **Frontend Dashboard:** React.js + Tailwind CSS.

- **API Layer:** FastAPI backend for event submission, scoring, reports, and administration.
- **Collision Scoring Service:** Python with Pandas / Scikit-learn utilities.
- **Data Ingestion Layer:** Scheduled jobs that collect and normalize events from selected APIs.
- **Storage Layer:** PostgreSQL for users, submitted events, external events, and reports; Redis optionally for caching.

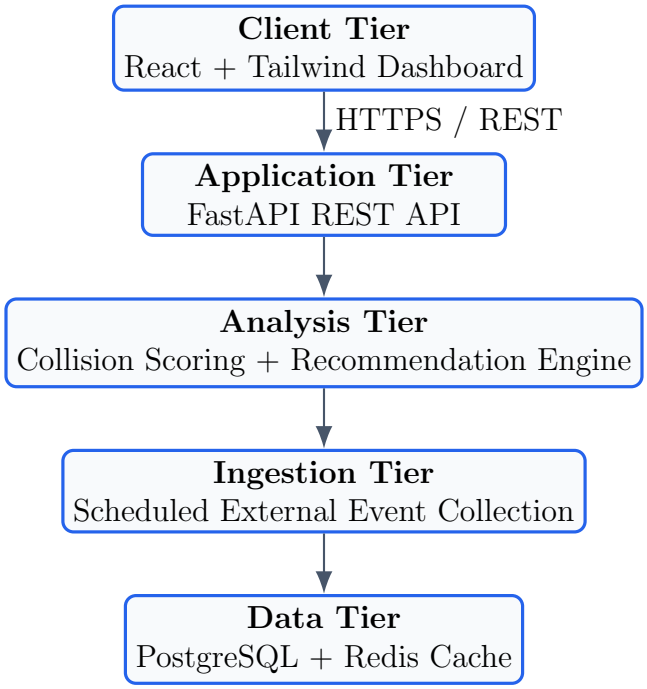

Figure 3: Event Collision Detector Layered Web Architecture

5.3 System Modeling: Use Cases and Functional Specifications

The primary actors are the *Event Organizer*, the *System Administrator*, and the *External Event Sources*.

ID	Use Case	Functional Specification & Scope
UC-1	Submit Proposed Event	User enters event date, industry, target audience, and supporting details.
UC-2	Collect External Events	System periodically retrieves and normalizes events from selected APIs and calendars.
UC-3	Compute Collision Score	Scoring service compares the proposed event with nearby external events.
UC-4	Display Risk Breakdown	Dashboard displays High / Medium / Low risk and the specific conflicting events.
UC-5	Recommend Alternative Dates	Recommendation engine evaluates nearby dates and returns lower-risk options.
UC-6	Generate Collision Report	System stores and displays an explainable report for each analysis.
UC-7	Maintain Event Data	Administrator updates invalid events and manages active data sources.

5.4 CI/CD and fast delivery enabler

CI/CD pipelines will run unit tests for scoring and normalization logic, verify frontend-backend API contracts, validate builds, and publish project documentation through GitHub Actions.

6 Evaluation Criterion

6.1 Primary evaluation metric

Collision Score Latency (CSL): Time from proposed-event submission to returned risk score and recommendation set.

- **Target:** Median CSL $\leq$ 500 ms against a pre-indexed event dataset.
- **Measurement:** Backend request timing recorded for each scoring request.

6.2 Secondary evaluation metrics

- **Scoring Agreement:** At least 80% agreement with manual assessment on a labeled validation set.
- **Ingestion Freshness:** Newly published events reflected within 24 hours of the ingestion cycle.
- **Recommendation Quality:** At least 90% of recommended dates should have a lower score than the original date.
- **Dashboard Responsiveness:** Risk details and recommendations render within one second after the API response.

6.3 Pilot validation plan

Test historical launch dates against manually researched competing events and conduct a small usability trial with students role-playing marketing or product managers.

7 Scalability

1. **Scalable theoretically:** Separate ingestion from scoring, cache repeated date windows, and introduce indexing as event volume grows.
2. **Operationally deployable practically:** Containerize frontend and backend, add industries through new source configurations, and run initially on standard relational storage with lightweight caching.

8 Engine availability heuristic

- **React + Tailwind CSS:** Frontend dashboard and calendar interface.
- **FastAPI:** Python backend for event and scoring endpoints.
- **PostgreSQL:** Storage for events, users, and reports.
- **Redis:** Optional cache for frequently requested date ranges.
- **Pandas / Scikit-learn:** Data preprocessing and scoring utilities.
- **External Event APIs:** Initial event ingestion sources.
- **GitHub Actions:** Automated testing and CI/CD.

9 Project scope and deliverables

9.1 Initial deliverable

- Basic React dashboard for event submission.
- One external event source integrated into the ingestion pipeline.
- Working FastAPI endpoint returning High / Medium / Low collision risk.
- Risk breakdown showing the events responsible for the score.
- PostgreSQL persistence for submitted events and reports.
- Automated CI pipeline for linting, testing, and build validation.

9.2 Subsequent deliverables

- Additional event sources and industry coverage.
- Alternative-date recommendation engine.
- Improved caching and query performance.
- Administrator interface for event-source management.
- Optional calendar integration for confirmed launch dates.

10 Risks and mitigations

- **API Rate Limits / Data Gaps:** Cache retrieved events and reduce confidence when source data is incomplete.
- **Scoring Heuristic Inaccuracy:** Validate weights against manually labeled historical examples.
- **Cold-Start Coverage:** Seed the event store with an initial historical batch.
- **Inconsistent Event Metadata:** Normalize incoming events into a common schema before scoring.
- **Over-Engineering:** Keep the first version limited to one or two industries and an explainable scoring function.

11 Summary

The **Event Collision Detector** is a deployable scheduling-intelligence platform that helps teams avoid launching into crowded attention windows. It combines external event aggregation, explainable collision scoring, and alternative-date recommendations in a single web dashboard. The project emphasizes rapid time-to-value, measurable evaluation criteria, modular architecture, mature tooling, and a clear path from prototype to scalable product.